



Predeposit Campaign Vault

Security Review



Disclaimer

Security Review

Predeposit Campaign Vault



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Predeposit Campaign Vault



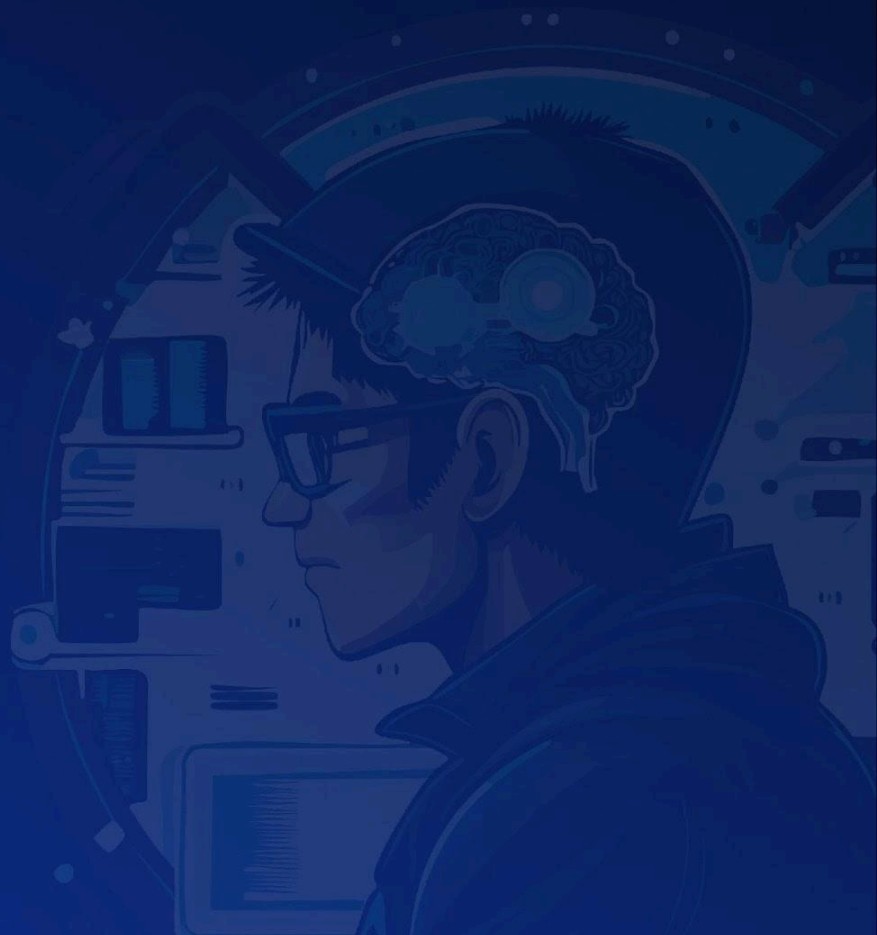
Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Vault-M01	18
3S-Vault-L01	20
3S-Vault-L02	21
3S-Vault-N01	23

Summary

Security Review

Predeposit Campaign Vault



Summary

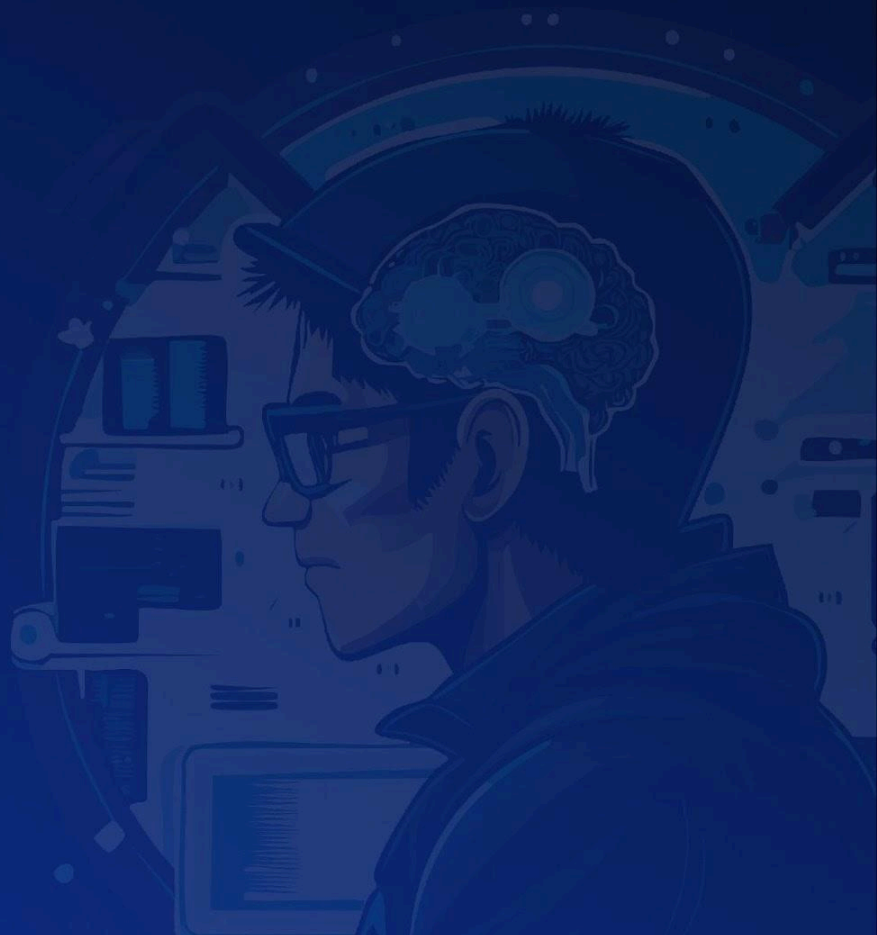
Three Sigma audited Keyring PredepositCampaign in a 0.8 person week engagement. The audit was conducted from 19/03/2026 to 20/03/2026.

Protocol Description

The Pre-Deposit Campaign Vault is an ERC4626-compatible smart contract that bootstraps initial USDC liquidity for the [un]wind RWA protocol. KYC-verified users (gated via Keyring on-chain credentials) deposit USDC during a fixed campaign window in exchange for non-transferable vault shares. Once the campaign deadline passes, a privileged operator migrates the collected USDC into an Euler v2 eVault, transitioning the vault to a redeemable state where depositors can claim their proportional eTokens. If the operator fails to migrate within an immutable grace period, a permissionless refund mechanism activates, allowing depositors to reclaim their original USDC. The vault enforces a strict, irreversible state machine (Open - Closed - SuccessRedeemable | FailedRefundable), immutable configuration parameters (hard cap, minimum deposit, deadline, grace period), and role-based access control via admin and operator multisigs.

Scope Security Review

Predeposit Campaign Vault



Scope

Filepath	nSLOC
src/PreDepositCampaignVault.sol	464
Total	464

Methodology

Security Review

Predeposit Campaign Vault



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

Predeposit Campaign Vault



Project Dashboard

Application Summary

Name	Keyring PredepositCampaign
Repository	https://github.com/Keyring-Network/keyring-smart-contracts-utils
Commit	1b1be2e
Language	Solidity
Platform	Avalanche

Engagement Summary

Timeline	19/03/2026 to 20/03/2026
Nº of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	1	0	1
Low	2	1	1
None	1	1	0

Category Breakdown

Suggestion	1
Documentation	0
Bug	2
Optimization	1
Good Code Practices	0

Risk Section Security Review

Predeposit Campaign Vault



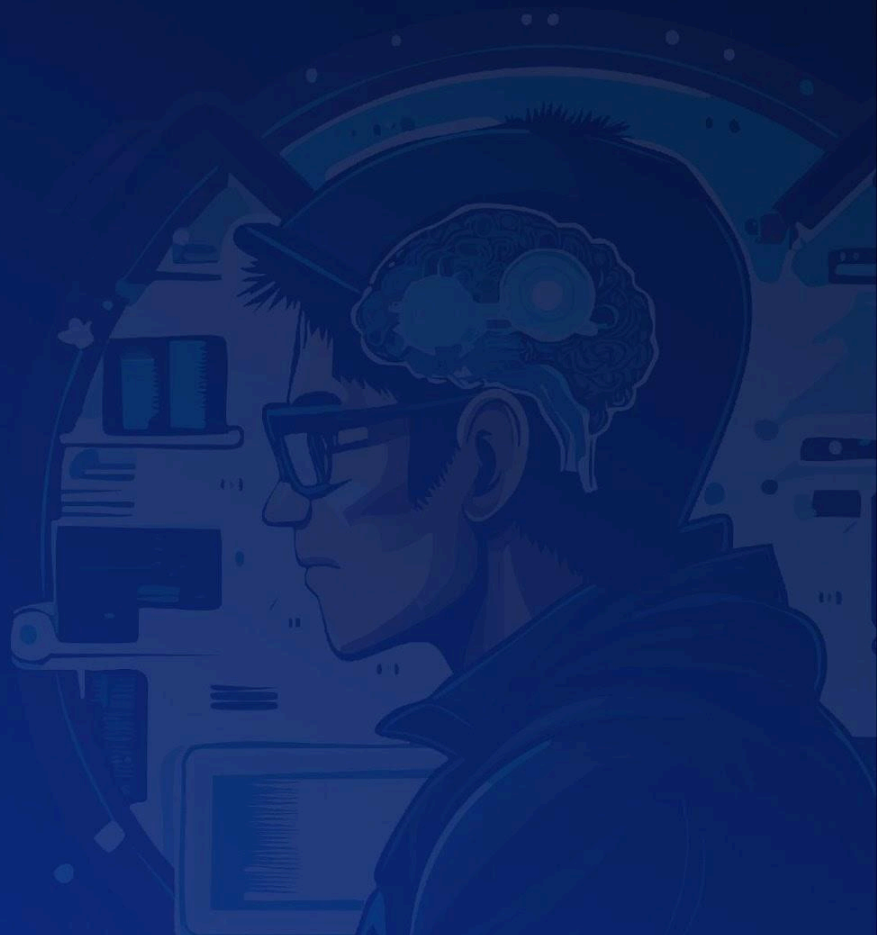
Risk Section

No risks were identified.

Findings

Security Review

Predeposit Campaign Vault



Findings

3S-Vault-M01

Keyring credential revocation after migration to **SuccessRedeemable** permanently locks user's eTokens

Id	3S-Vault-M01
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

When the campaign transitions to **SuccessRedeemable**, users can redeem their vault shares for eTokens through **redeem()** or **withdraw**. Both functions enforce a Keyring credential check via **maxRedeem** / **maxWithdraw**:

```
function maxRedeem(address _owner) public view override returns (uint256) {  
    if (_currentState() != CampaignState.SuccessRedeemable) { return 0; }  
    if (!i_keyring.checkCredential(_owner, i_policyId)) { return 0; }  
    return balanceOf(_owner);  
}
```

If a user's KYC credential is revoked after the campaign enters **SuccessRedeemable** (but before the user redeems), **maxRedeem** and **maxWithdraw** return 0, causing **redeem** and **withdraw** to revert:

```
// redeem (line 189-191)  
if (_shares > maxRedeem(_owner)) {  
    revert ERC4626ExceededMaxRedeem(_owner, _shares, maxRedeem(_owner));  
}
```

The **KEYRING_OPERATOR_ROLE** can call **redeem/withdraw** on behalf of the user , but the credential check is on the **_owner** address - not the caller. So even the operator cannot bypass this.

The failure path:

1. User passes KYC, deposits USDC during **Open** state
2. Campaign closes, operator calls **depositIntoEuler** -> state becomes **SuccessRedeemable**
3. User's Keyring credential is revoked (KYC expiry, sanctions, policy change)
4. User calls **redeem** -> reverts because **maxRedeem()** returns 0
5. Operator calls **redeem** on user's behalf -> still reverts (credential check is on **_owner**)
6. No path exists to recover the eTokens - they are locked permanently

Recommendation

Add an operator-controlled emergency redemption that bypasses the credential check in **SuccessRedeemable** state, or remove the credential check from the redemption path entirely since the user was already verified at deposit time.

3S-Vault-L01

Use of Revert Strings Instead of Custom Error

Id	3S-Vault-L01
Classification	Low
Impact	Low
Likelihood	Low
Category	Optimization
Status	Acknowledged

Description

Across the **IRMLogisticBase.sol** and **IRMLogisticTwisted.sol** files, require statements with string messages are used instead of custom errors. Since Solidity ^0.8.4 supports custom errors, replacing them with custom errors can reduce both runtime revert gas and deployment cost by avoiding string allocation/storage.

Recommendation

Replace **require** statements to custom errors to save gas on each function call and deployment.

3S-Vault-L02

Missing CampaignClosed event on early operator refund activation

Id	3S-Vault-L02
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in fd6d6fa .

Description

When the operator calls **activateRefundMode** before the campaign deadline, the function first calls **_transitionToClosedIfNeeded**. However, this helper is a no-op when **block.timestamp < i_campaignDeadline**:

```
// Line 353-362
function _transitionToClosedIfNeeded() internal {
    if (s_state != CampaignState.Open || block.timestamp < i_campaignDeadline) {
        return; // <-- no-op, no CampaignClosed emitted
    }
    s_state = CampaignState.Closed;
    s_closedTimestamp = i_campaignDeadline;
    emit CampaignClosed(msg.sender, _totalDeposited(), s_closedTimestamp);
}
```

Since the operator has **KEYRING_OPERATOR_ROLE**, the check at line 245 passes and **_activateRefundMode()** is called directly. This function transitions the state from **Open** straight to **FailedRefundable**, sets **s_closedTimestamp = block.timestamp**, and emits only **RefundModeActivated**:

```
// Line 366-373
function _activateRefundMode(address _activatedBy) internal {
    if (s_closedTimestamp == 0) {
        s_closedTimestamp = block.timestamp;
    }
    s_state = CampaignState.FailedRefundable;
    emit RefundModeActivated(_activatedBy);
}
```

```
}
```

The **Open -> Closed** transition is completely skipped - no **CampaignClosed** event is emitted. This breaks PRD requirement F-32:

> All state transitions emit events. Full lifecycle reconstructable from on-chain events alone.

Any off-chain system relying on **CampaignClosed** to detect the intermediate state transition will see the campaign jump from **Open** directly to **FailedRefundable** with no explanation.

Recommendation

Emit **CampaignClosed** inside `_activateRefundMode()` when the campaign is still in **Open** state, before transitioning to **FailedRefundable**.

3S-Vault-N01

Lack of slippage validation in **depositIntoEuler**

Id	3S-Vault-N01
Classification	None
Category	Suggestion
Status	Addressed in 7b0a835 .

Description

After users deposit **USDC** during the campaign's **open** state, if the block timestamp passes the campaign deadline, the **KEYRING_OPERATOR_ROLE** will deposit the collected **USDC** into Euler via **depositIntoEuler** in exchange for eTokens. During this call, the operator admin may accidentally provide a zero value for **_minETokensExpected**. Because the contract does not validate the slippage parameter, the slippage protection is silently disabled.

Recommendation

Validate **_minETokensExpected** to ensure it is always a non-zero value or document this if it is an intended design.